User System Requirements Document (USRD): Sentiment Intelligence Platform

Version 1.0

Kushal Trivedi
M.S. Data Science
Arizona State University, Tempe, AZ, USA
kushaltrivedi82@gmail.com

***Abstract*—This document defines the system requirements for the Sentiment Intelligence Platform—an end-to-end NLP project that fine-tunes DistilBERT on the Yelp Review Full dataset for five-class sentiment classification. The platform covers nine sequential processes: data ingestion, exploratory data analysis, model training, evaluation, explainability, API deployment, front-end visualization, drift monitoring, and CI/CD deployment. This USRD specifies objectives, data requirements, per-process inputs and outputs, technology stack, hardware constraints, and risk mitigations.**

***Index Terms*—Sentiment Analysis, DistilBERT, NLP, MLOps, FastAPI, SHAP, Evidently, HuggingFace, DVC, MLflow**

I. Introduction

Sentiment analysis on customer reviews is a well-studied NLP problem, but most academic implementations stop at model training. This project builds a complete production pipeline—from raw Yelp data to a deployed, monitored API with a React dashboard and token-level SHAP explainability.

The platform targets five sentiment classes (1–5 stars) using `distilbert-base-uncased` fine-tuned on a stratified 50,000-sample subset of Yelp reviews. Experiments are tracked with MLflow, data is versioned with DVC, predictions are served via FastAPI, and concept drift is monitored with Evidently.

A. Motivation

Most public sentiment models are binary classifiers. A five-class classifier on Yelp data is more realistic: adjacent classes (e.g., 3-star vs. 4-star) are genuinely ambiguous, which makes the problem harder. The per-class confusion patterns also give meaningful material for SHAP-based error analysis.

B. Scope

This document covers:

- Project objectives and measurable success criteria
- Dataset specifications with exact split sizes
- Nine-process architecture with per-process I/O
- Technology stack and hardware constraints
- Risk register with mitigations

II. Project Objectives

A. Primary Objectives

1) Fine-tune DistilBERT for five-class sentiment on Yelp reviews.
2) Beat a TF-IDF + Logistic Regression baseline by $\geq$10 macro F1 points.
3) Deploy a real-time inference API accessible to external users.
4) Provide token-level SHAP attribution for every prediction.
5) Detect prediction and data drift in production via Evidently.

B. Secondary Objectives

1) Demonstrate reproducibility via MLflow and DVC.
2) Build a recruiter-accessible React dashboard on HuggingFace Spaces.
3) Maintain a clean, modular codebase with full CI/CD coverage.

C. Success Criteria

TABLE I
Measurable Success Criteria

Metric	Baseline	Target
Macro F1 (test)	$\sim$0.52	$\geq$0.62
Weighted F1	$\sim$0.52	$\geq$0.62
API latency (1 req)	–	$<$300 ms
API latency (10 req)	–	$<$1 s
SHAP coverage	–	100%
Drift report	–	Weekly HTML

III. Dataset Specification

A. Source

The project uses `yelp_review_full` from HuggingFace Datasets [1], compiled by Zhang et al. [2] from the Yelp Dataset Challenge.

TABLE II
YELP REVIEW FULL DATASET STATISTICS

Split	Samples	Per Class
Train (full)	650,000	130,000
Test (full)	50,000	10,000
Total	**700,000**	**140,000**
After stratified 50k sampling (used):		
Train	39,968	≈7,994
Validation	4,996	≈999
Test	4,996	≈999

B. Statistics

C. Label Schema

Labels are integer-encoded as $\{0, 1, 2, 3, 4\}$ mapping to $\{1\star, 2\star, 3\star, 4\star, 5\star\}$ Yelp ratings. The dataset is perfectly balanced (140k per class in full data), so class weights computed on the 50k sample are all ≈ 1.0.

D. Data Quality

The cleaning step: (1) strips leading/trailing whitespace, (2) collapses multi-space sequences, (3) removes non-ASCII characters, and (4) drops reviews shorter than 10 characters. In the 50k stratified sample, 40 records were removed.

IV. System Architecture

The platform is structured as nine sequential processes. Each process has defined inputs, outputs, and a verification step before the next process begins. Fig. 1 shows the full pipeline.

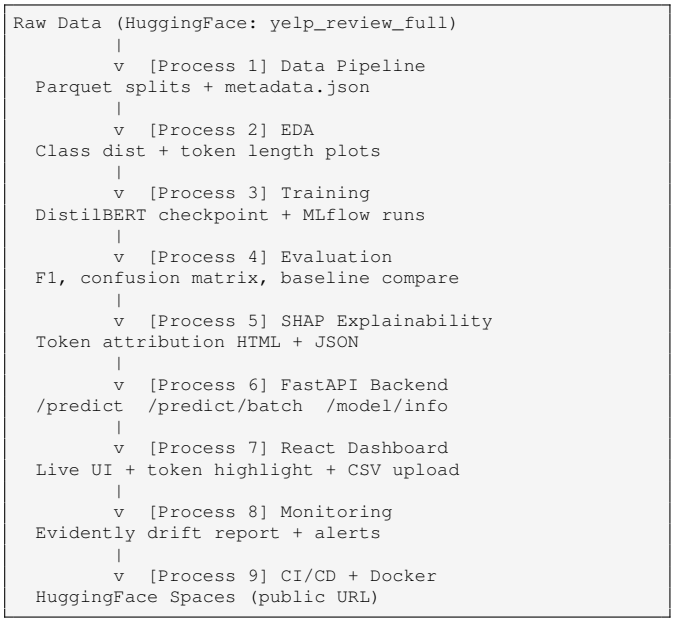


Fig. 1. Nine-process pipeline architecture

V. Process Requirements

A. Process 1: Data Pipeline [Status: Complete]

Objective: Download Yelp data, apply stratified sampling, clean text, split into train/val/test, and persist artifacts.

Inputs:

- HuggingFace dataset: `yelp_review_full`
- Config file: `configs/config.yaml`

Steps:

1) Download 700k records via `load_dataset()`.
2) Rename: `text`→`review_text`, `label`→`sentiment`.
3) Stratified sample: 50,000 records (10k per class).
4) Clean text: strip whitespace, remove non-ASCII, drop <10 chars.
5) Two-stage stratified split: 80% train / 10% val / 10% test.
6) Compute class weights via `compute_class_weight("balanced")`.
7) Save splits as Parquet; metadata as JSON.

Outputs:

- `data/splits/train.parquet` 39,968 rows
- `data/splits/val.parquet` 4,996 rows
- `data/splits/test.parquet` 4,996 rows
- `data/splits/metadata.json` sizes, weights, dist.

Verification: All files present; uniform label distribution; no null values in `review_text` or `sentiment`.

B. Process 2: EDA [Status: In Progress]

Objective: Understand data distribution and validate tokenization assumptions before training.

Required Plots:

1) Class distribution across train/val/test splits.
2) Character length by class: box plot and KDE overlay.
3) DistilBERT token length histogram with `max_length=256` marked.
4) Mean/median character length heatmap per class.

Outputs:

- `reports/eda/*.png` (four plots)
- `reports/eda/eda_summary.json` (P50, P90, P95 token lengths; coverage at `max_length`)

Key Decision: If P90 token length exceeds 256, `max_length` in config will be updated before training.

C. Process 3: Model Training [Status: Pending]

Objective: Fine-tune `distilbert-base-uncased` for five-class classification with class-weighted loss, MLflow tracking, and early stopping.

Layer Freezing: DistilBERT has six transformer blocks (0–5). Blocks 0–3 and embeddings are frozen; only blocks 4–5 and the classification head are updated. This reduces trainable parameters from 66M to ≈15M.

MLflow Tracking: Each run logs: learning rate, batch size, max_length, train/val loss, macro F1, and accuracy per epoch. Best checkpoint (by val macro F1) is saved to `models/distilbert-sentiment/best/`.

TABLE III
TRAINING HYPERPARAMETERS

Parameter	Value
Base model	distilbert-base-uncased
Num labels	5
Max token length	256
Frozen layers	Embeddings + blocks 0–3
Trainable layers	Blocks 4–5 + classifier head
Trainable params	≈15M of 66M total
Epochs	5 (early stopping patience = 2)
Batch size	32
Learning rate	2e-5
Weight decay	0.01
Warmup ratio	0.1
Optimizer	AdamW
Device	MPS (Apple Silicon M-series)
Loss	CrossEntropyLoss (class-weighted)

D. Process 4: Evaluation [Status: Pending]

Objective: Evaluate best checkpoint on held-out test set and compare against TF-IDF + Logistic Regression baseline.

Metrics: Accuracy, macro F1, weighted F1, macro ROC-AUC (one-vs-rest), per-class precision/recall/F1.

Outputs:

- Normalized confusion matrix (both models)
- Bar chart: side-by-side metric comparison
- `reports/evaluation/evaluation_results.json`
- `reports/evaluation/classification_report.csv`

E. Process 5: SHAP Explainability [Status: Pending]

Objective: Generate token-level SHAP attributions using the Partition Explainer for transformer-based text classifiers [4].

Method: `shap.Explainer` with `shap.maskers.Text` is applied to 10 test samples (two per class). SHAP values are computed for the predicted class; positive values push toward the prediction, negative values push away.

Outputs:

- Per-sample HTML token highlight visualization
- Per-sample JSON with signed token importance list
- Aggregate bar chart: top-20 tokens by cumulative SHAP magnitude

F. Process 6: FastAPI Backend [Status: Pending]

Objective: Serve the trained model as a REST API.

Model is loaded once at startup as a singleton. Predictions are logged to SQLite for monitoring. CORS is enabled for the React dashboard.

G. Process 7: React Dashboard [Status: Pending]

Objective: Browser-accessible UI for single and batch sentiment analysis.

Features:

TABLE IV
API ENDPOINTS

Method	Endpoint	Description
POST	`/predict`	Single text + optional SHAP
POST	`/predict/batch`	Bulk inference (max 100)
GET	`/model/info`	Version, metrics, count
GET	`/health`	Liveness check

- Text input with real-time prediction and confidence bar.
- Token highlight visualization (green = positive SHAP, red = negative).
- CSV batch upload with downloadable results table.

H. Process 8: Monitoring [Status: Pending]

Objective: Detect data drift and confidence degradation in prediction logs.

Method: Prediction logs are split chronologically (first half = reference, second half = current). Evidently's `DataDriftPreset` and `TargetDriftPreset` run on three numeric features: character length, confidence score, predicted class.

Outputs:

- Timestamped HTML drift report
- `reports/drift/monitoring_summary.json` with drift flag and confidence alert

I. Process 9: CI/CD and Deployment [Status: Pending]

Objective: Automate testing, Docker build, and deployment to HuggingFace Spaces on every push to `main`.

GitHub Actions Pipeline:

1) **Job 1 — Test:** flake8 lint + pytest.
2) **Job 2 — Docker:** build image, run container, check `/health`.
3) **Job 3 — Deploy:** push to HuggingFace Spaces via git remote.

VI. TECHNOLOGY STACK

VII. HARDWARE CONSTRAINTS

Training runs on a MacBook Air with Apple Silicon (M-series). There is no CUDA GPU; PyTorch's MPS backend is used instead.

Known MPS Limitations:

- No `fp16` mixed precision; training uses `fp32`.
- Some SHAP operations fall back to CPU automatically.
- `DataLoader num_workers` must be kept low to avoid MPS memory errors.

Mitigations:

- Dataset capped at 50k samples (training under 2 hours on MPS).
- Layer freezing reduces trainable params from 66M to ≈15M.
- SHAP inference explicitly runs on CPU; model inference uses MPS.

TABLE V
TECHNOLOGY STACK

Layer	Technology
Model	DistilBERT (HuggingFace Transformers [3])
Framework	PyTorch (MPS backend)
Data versioning	DVC
Experiment tracking	MLflow [5]
Explainability	SHAP (Partition Explainer)
Backend	FastAPI + Pydantic + Uvicorn
Frontend	React + TailwindCSS
Monitoring	Evidently
Containerization	Docker
CI/CD	GitHub Actions
Deployment	HuggingFace Spaces
Storage	SQLite (logs), Parquet (data)

VIII. RISK REGISTER

TABLE VI
RISK REGISTER

Risk	Likelihood	Mitigation
MPS training crash	Medium	Reduce batch size; fall back to CPU
SHAP too slow	Medium	Limit to 10 samples; cache JSON
HuggingFace rate limit	Low	Set `HF_TOKEN` env var
Model below baseline	Low	Unfreeze layers 2–3; add epochs
SQLite write contention	Low	Switch to PostgreSQL in production

IX. DIRECTORY STRUCTURE

```
sentiment-intelligence-platform/
+-- configs/
|   +-- config.yaml
+-- src/
|   +-- pipeline/
|   |   +-- data_pipeline.py  [P1: complete]
|   |   +-- eda.py            [P2: in progress]
|   +-- training/
|   |   +-- train.py          [P3]
|   +-- evaluation/
|   |   +-- evaluate.py       [P4]
|   +-- explainability/
|   |   +-- shap_explainer.py [P5]
|   +-- api/
|   |   +-- main.py           [P6]
|   +-- monitoring/
|       +-- drift_monitor.py  [P8]
+-- frontend/                 [P7]
+-- data/splits/
|   +-- train.parquet         39,968 rows
|   +-- val.parquet            4,996 rows
|   +-- test.parquet           4,996 rows
|   +-- metadata.json
+-- models/
+-- reports/
+-- tests/
+-- .github/workflows/
|   +-- ci_cd.yml             [P9]
+-- Dockerfile
+-- requirements.txt
```

X. CONCLUSION

This document specifies the full requirements for the Sentiment Intelligence Platform. Process 1 (data pipeline) is complete and verified. Processes 2–9 follow sequentially, each producing artifacts that feed the next stage. Together they demonstrate end-to-end ML engineering across NLP modeling, explainability, API development, drift monitoring, and MLOps tooling on Apple Silicon hardware.

REFERENCES

[1] Yelp Inc., "Yelp Review Full Dataset," HuggingFace Datasets, 2015. [Online]. Available: https://huggingface.co/datasets/Yelp/yelp_review_full

[2] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. NeurIPS*, 2015, pp. 649–657.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017.

[5] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.